MSAS – Assignment #1: Simulation

Alejandro Rivera Míguez, 276270

1 Implicit equations

Exercise 1

Consider the physical model of a mechanical system made of four rigid rods $\mathbf{a}_i$ with $i \in [1;4]$ in a kinematic chain, as shown in Fig. 1. The system has only two degrees of freedom. The objective is to determine, fixing the angle β, the corresponding angle of equilibrium α between the rods $\mathbf{a}_1$ and $\mathbf{a}_2$. Writing the kinematic closure, it is possible to obtain the following equation

$$\frac{a_1}{a_2}\cos\beta - \frac{a_1}{a_4}\cos\alpha - \cos(\beta - \alpha) = -\frac{a_1^2 + a_2^2 - a_3^2 + a_4^2}{2a_2a_4} \tag{1}$$

where a_i is the length of the corresponding rod. This equation, called the equation of Freudenstein, can be clearly re-written in the form $f(\alpha) = 0$ and solved, once β is fixed, for a particular value of α. Assume that $a_1 = 10$ cm, $a_2 = 13$ cm, $a_3 = 8$ cm, and $a_4 = 10$ cm and

1) Implement a general-purpose Newton solver (NS).
2) Solve Eq. (1) with NS for $\beta \in [0, \frac{2}{3}\pi]$ with the analytical derivative of $f(\alpha)$ and tolerance set to 10^{-5}. For every β value, use two initial guesses, -0.1 and $\frac{2}{3}\pi$. What do you observe? You can validate your results with the Matlab© built-in function `fzero`.
3) Repeat point 2) using the derivative estimated through finite differences and compare the accuracy of the two methods.
4) Eq. (1) can be solved analytically only for specific values of β. Find at least one and compare the analytical solution with yours.
5) Repeat point 2) extending the β interval up to π. What do you observe?

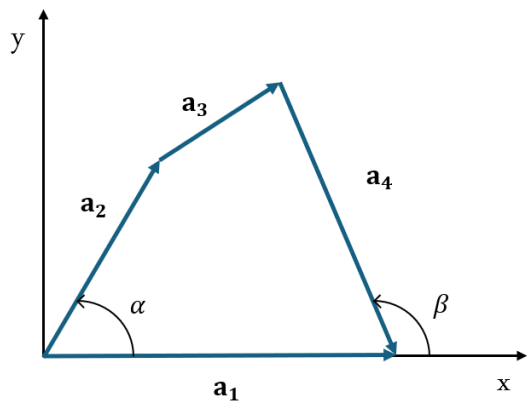


Figure 1: Kinematic chain.

(4 points)

1. General-purpose Newton solver (NS)

```
% Newton-Raphson method to solve equations of the form f(x) = 0
% Inputs:
%   f: function we want to find the root of, depends on x and optional parameters
%   df: derivative of the function f with respect to x
%   x0: initial guess for the Newton-Raphson iteration
%   tol: tolerance for convergence
%   max_iter: maximum number of allowed iterations
%   varargin: other optional parameters for f and df
% Output:
%   x: approximate root of the function

function x = newton_solver(f, df, x0, tol, max_iter, varargin)
    x = x0;  % Initialize the variable
    for i = 1:max_iter
        f_val = f(x, varargin{:});     % Evaluate f at x with additional parameters
        df_val = df(x, varargin{:});   % Evaluate df at x with additional parameters

        % Update x using Newton-Raphson
        x_new = x - f_val / df_val;

        % Check for convergence
        if abs(x_new - x) < tol
            x = x_new;
            return;
        end
        x = x_new;
    end
    % If the method does not converge, a warning message is returned
    warning('Convergence not achieved within the maximum number of iterations.');
end
```

2. Solve Eq. (1) with NS for $\beta \in [0, \frac{2}{3}\pi]$ with the analytical derivative of $f(\alpha)$ and tolerance set to 10^{-5}. For every β value, use two initial guesses, -0.1 and $\frac{2}{3}\pi$. What do you observe? You can validate your results with the Matlab© built-in function `fzero`.

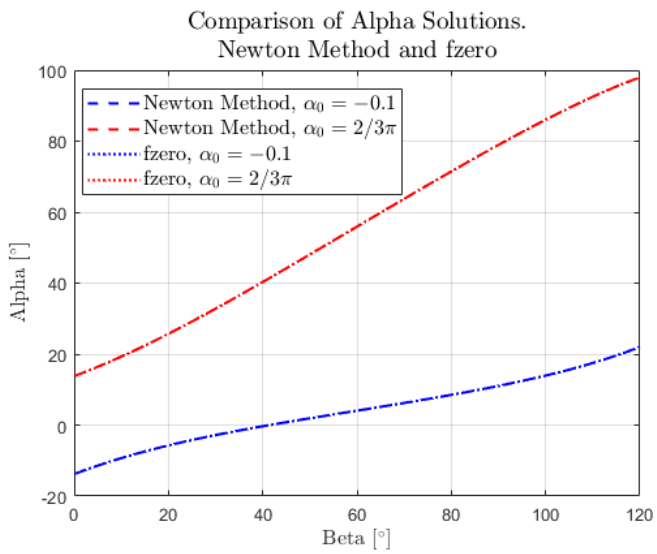


Figure 2: Comparison of alpha solutions. Newton method vs fzero

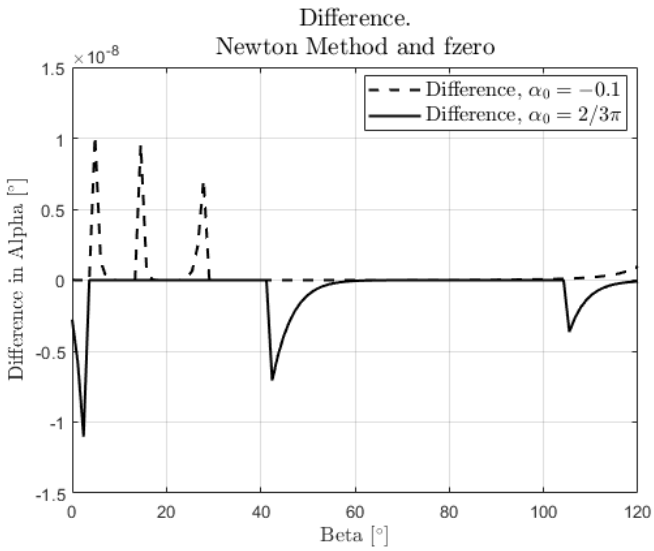


Figure 3: Difference. Newton method vs fzero.

We can appreciate:

- **Figure 2.** demonstrates that solutions for α vary based on initial guesses ($\alpha_0 = -0.1$ and $\alpha_0 = \frac{2}{3}\pi$). This suggests the equation has multiple solutions for specific β values, with the initial guess α_0 determining the solution convergence. Therefore, selecting an appropriate starting point is critical.
- **Figure 3.** Both Newton's method and `fzero` are reliable, as the differences between them are minimal (on the order of 10^{-8}).
- The presence of multiple solutions for some β values is a significant characteristic of this equation, which should be considered in future analysis or applications.

3. Repeat point 2) using the derivative estimated through finite differences and compare the accuracy of the two methods.

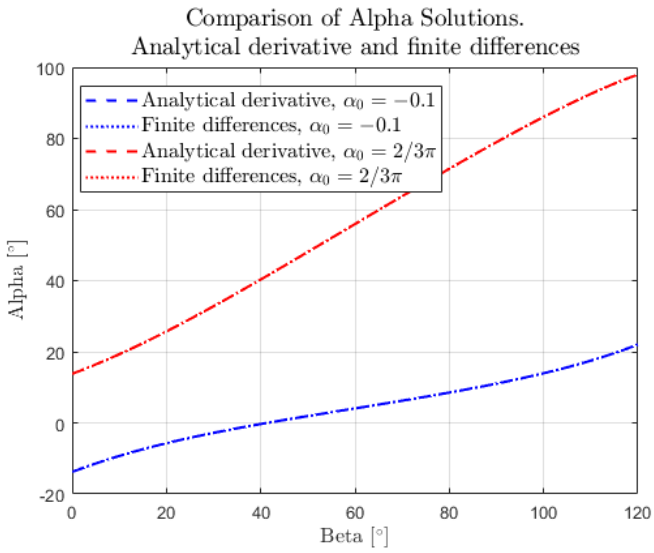


Figure 4: Comparison of Alpha Solutions using Newton method. Analytical derivative vs finite differences

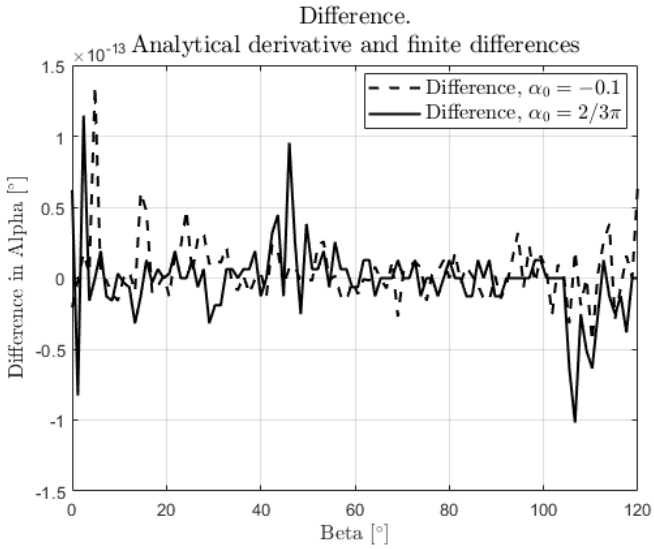


Figure 5: Difference in Newton method. Analytical derivative and finite differences.

The finite difference method used here is the centered difference approach, which can be represented as:

$$f'(\alpha) \approx \frac{f(\alpha + h) - f(\alpha - h)}{2h}$$

where h is a small increment. This formula approximates the derivative of a function f at point α by averaging the slopes from both sides of α, enhancing accuracy compared to one-sided differences.

We can see that:

- **Figure 4:** The solutions for α are calculated using both the analytical derivative and finite differences for initial values $\alpha_0 = -0.1$ and $\alpha_0 = \frac{2}{3}\pi$. Both methods yield nearly identical α values across a range of β, confirming the accuracy of the finite difference approximation.
- **Figure 5:** The differences between the α solutions obtained from the two methods are minimal (on the order of 10^{-13}), demonstrating the precision of the finite difference method.
- **Conclusion:** The finite difference method is validated as an accurate and reliable alternative to the analytical derivative, particularly when the latter is difficult or infeasible to compute.

4. Eq. (1) can be solved analytically only for specific values of β. Find at least one and compare the analytical solution with yours.

   Below are the analytical solutions for three cases:

   - Case $\beta = 0$

$$\alpha = \arccos\left(\frac{\frac{a_1^2+a_2^2-a_3^2+a_4^2}{2a_2a_4} + \frac{a_1}{a_2}}{\frac{a_1}{a_4} + 1}\right) \tag{2}$$

   - Case $\beta = \frac{\pi}{2}$

$$\alpha = \arctan\left(\frac{a_4}{a_1}\right) \pm \arccos\left(\frac{\frac{a_1^2+a_2^2-a_3^2+a_4^2}{2a_2a_4}}{\sqrt{\left(\frac{a_1}{a_4}\right)^2 + 1}}\right). \tag{3}$$

- Case $\beta = \alpha$

$$\alpha = \arccos\left(\frac{1 - \frac{a_1^2+a_2^2-a_3^2+a_4^2}{2a_2a_4}}{\frac{a_1}{a_2} - \frac{a_1}{a_4}}\right) \tag{4}$$

Table 1: Comparison of Analytical and Numerical Solutions for Specific Cases

Case	β (radians)	α Analytical (radians)	α Numerical (radians)	**Difference**
1	0	0.24077	0.24077	4.8388×10^{-11}
2	$\frac{\pi}{2}$	1.3780	1.3780	2.2204×10^{-16}
3	$\beta = \alpha$	0.72273	0.72273	2.6799×10^{-8}

The differences between the analytical and numerical solutions are extremely small. These minimal discrepancies confirm the accuracy of the analytical solutions and validate that the Newton method correctly converges to the expected values.

5. Repeat point 2) extending the β interval up to π. What do you observe?

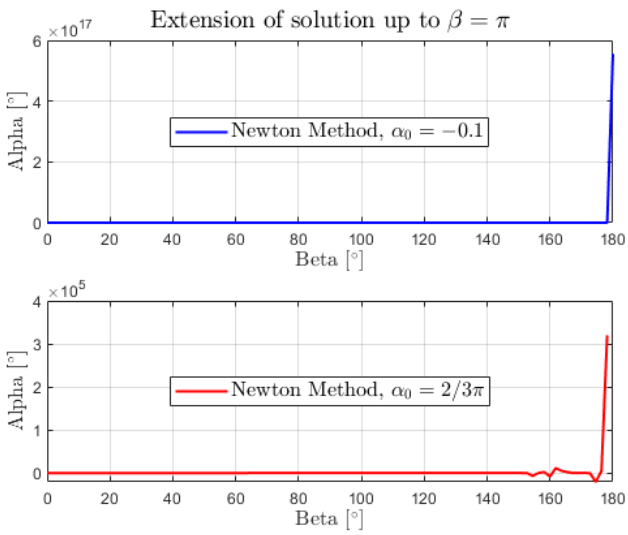


Figure 6: Extension of the solution up to $\beta = \pi$.

As β approaches π, the value of α diverges, indicating that the Newton method fails to converge. Analytically, no finite solution for α exists at $\beta = \pi$:

$$\alpha = \arccos\left(\frac{\frac{a_1^2+a_2^2-a_3^2+a_4^2}{2a_2a_4} - \frac{a_1}{a_2}}{\frac{a_1}{a_4} - 1}\right).$$

MATLAB returned an infinite value for α at $\beta = \pi$, expressed as:

$$\text{For } \beta = \pi: \quad \alpha = 0 + \infty i \text{ radians.}$$

This result indicates that Eq. (1) lacks a real, finite solution at this point.

As a conclusion, the divergence observed in Figure 6 aligns with analytical findings. Extending the β interval up to π highlights the limitations of Eq. (1) and reveals the points where the Newton method fails due to the equation's inherent structure.

2 Numerical solution of ODE

Exercise 2

The physical model of a moving object with decreasing mass is shown in Fig. 7. The mathematical model of the system is expressed by the equations:

$$m(t)\frac{dv}{dt} = \overline{F} + f(t) - \alpha \cdot v \tag{5}$$

$$m(t) = m_0 - c_m \cdot t \tag{6}$$

$$\overline{F} = -\,0.5 \cdot \rho \cdot C_d \cdot A_m \cdot v^2 \tag{7}$$

Assuming that: $v(0) = 0$ [m/s], $m_0 = 20$ [kg], $c_m = 0.1$ [kg/s], $f(t) = 1$ [N], $\alpha = 0.01$ [Ns/m], $\rho = 0$ [kg/m^3], and Eq. (5) has the exact solution:

$$v(t) = \frac{f(t)}{\alpha} - \left[\frac{f(t)}{\alpha} - v(0)\right]\left[1 - \frac{c_m \cdot t}{m_0}\right]^{\frac{\alpha}{c_m}} \tag{8}$$

Answer to the following tasks:

1) Implement a general-purpose, fixed-step Heun's method (RK2);
2) Solve Eq. (5) using RK2 in $t \in [0, 160]$s for $h_1 = 50$, $h_2 = 20$, $h_3 = 10$, $h_4 = 1$ and compare the numerical vs the analytical solution;
3) Repeat points 1)–2) with RK4;
4) Trade off between CPU time & integration error.

Now, assuming that: the fluid density is $\rho = 900$ [kg/m^3], $C_d = 2.05$, and $A_m = 1$ [m^2], answer to the following tasks:

1) Solve Eq. (5) using RK2 in $t \in [0, 160]$s for $h_1 = 1$;
2) From the results of the previous point, use a proper ode of Matlab to solve Eq. (5) ;
3) Discuss the results.

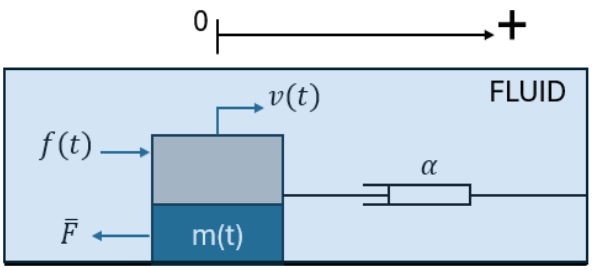


Figure 7: Moving object with decreasing mass.

(5 points)

1. Implement a general-purpose, fixed-step Heun's method (RK2);

```
% HEUN_METHOD implements Heun's method (a type of Runge-Kutta method) for solving ODEs
% Inputs:
%   fun - function handle representing the ODE (dx/dt = fun(t, x))
%   tspan - time span [t0, tf] over which to integrate
%   x0 - initial value of x at t0
%   h - step size for the integration
% Outputs:
%   t - vector of time values from t0 to tf
%   x - vector of solution values corresponding to each time step

function [t, x] = heun_method(fun, tspan, x0, h)
t0 = tspan(1);    % Initial time
tf = tspan(2);    % Final time
t = t0:h:tf;      % Time vector from t0 to tf with step size h
n = length(t);    % Number of time steps
x = zeros(1, n); % Pre-allocate solution vector for x
x(1) = x0;        % Set the initial condition

% Loop through time steps and apply Heun's method
    for k = 1:(n-1)
    k1 = fun(t(k), x(k)); % Compute the first slope estimate
    xP = x(k) + h * k1;   % Predict the value of x at the next step
    k2 = fun(t(k+1), xP); % Compute the second slope estimate using the predicted value
    x(k+1) = x(k) + (h/2) * (k1 + k2); % Update the solution using the average of the two slopes
    end
end
```

2. Solve Eq. (5) using RK2 in $t \in [0, 160]$s for $h_1 = 50$, $h_2 = 20$, $h_3 = 10$, $h_4 = 1$ and compare the numerical vs the analytical solution;

   The analytical solution for the case $\rho = 0$ serves as a reference to evaluate the accuracy of the numerical results obtained.

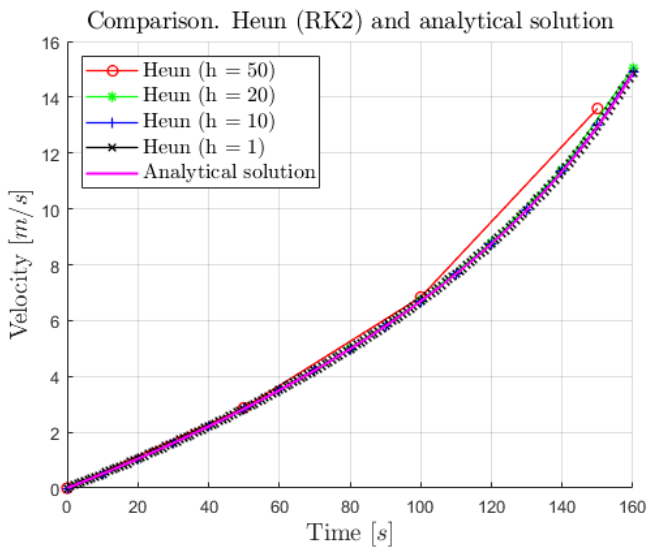


Figure 8: Comparison. Heun method (RK2) vs analytical solution.

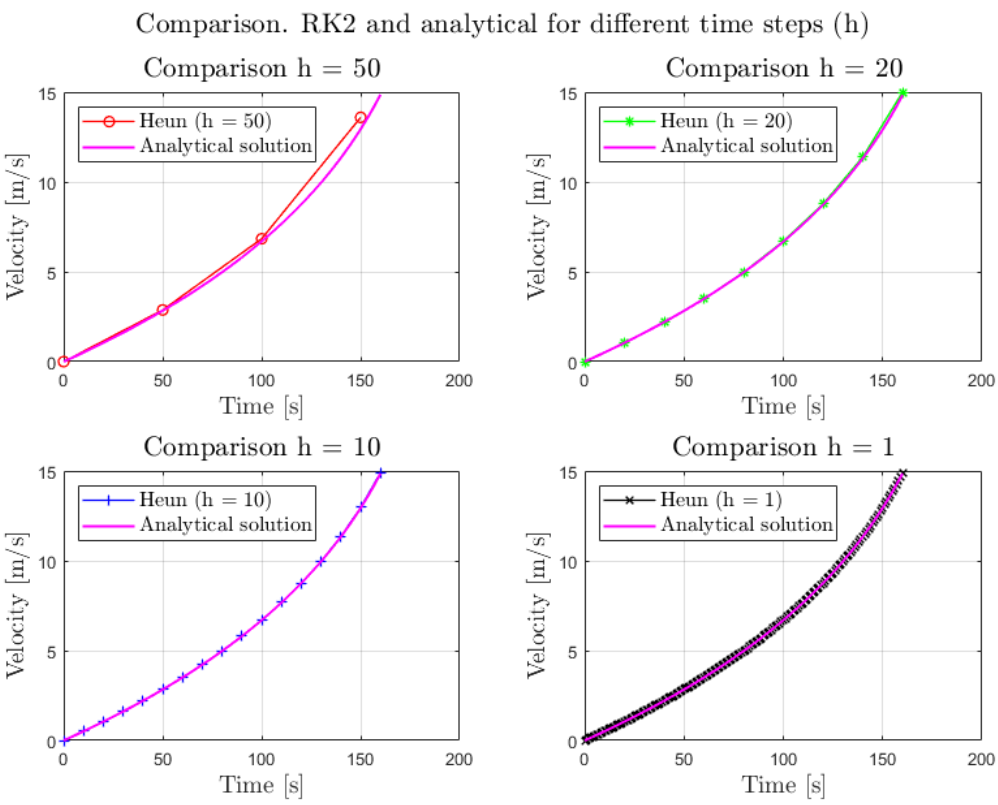


Figure 9: Comparison. Heun method (RK2) vs analytical solution for different time steps.

- **Figure 8:** This figure illustrates the Heun (RK2) solutions for different time steps overlaid with the analytical solution. The magenta curve represents the analytical solution and serves as the baseline for accuracy evaluation.

  - For **large time steps** (e.g., $h = 50$), significant deviations from the analytical solution are evident, especially as t increases, indicating that $h = 50$ is too large for accurate system dynamics.

  - As the **time step decreases** (e.g., $h = 1$), the RK2 numerical solution converges toward the analytical curve, demonstrating improved accuracy with smaller steps.

- **Figure 9:** This figure provides a detailed comparison of RK2 accuracy for varying h values against the analytical solution.

Table 2: Accuracy Comparison of the RK2 Solution for Different Time Steps h

Time Step h	Observations	Accuracy Level
50	Significant deviation from analytical solution	Low
20	Improved accuracy but still with deviations	Moderate
10	Minor discrepancies	High
1	Excellent agreement with analytical solution	Very High

Conclusion: The Heun method (RK2) achieves accurate numerical approximations of the analytical solution as the time step h decreases. Large time steps (e.g., $h = 50$) introduce significant errors, whereas smaller time steps (e.g., $h = 1$) yield solutions closely matching the analytical curve. This highlights the necessity of selecting an appropriate time step in numerical methods to ensure high accuracy when solving differential equations.

3. Repeat points 1)–2) with RK4;

```
% RK4_METHOD implements the fourth-order Runge-Kutta method for solving ODEs
% Inputs:
%   fun - function handle representing the ODE (dx/dt = fun(t, x))
%   tspan - time span [t0, tf] over which to integrate
%   x0 - initial value of x at t0
%   h - step size for the integration
% Outputs:
%   t - vector of time values from t0 to tf
%   x - vector of solution values corresponding to each time step

function [t, x] = rk4_method(fun, tspan, x0, h)
t0 = tspan(1);        % Initial time
tf = tspan(2);        % Final time
t = t0:h:tf;          % Time vector from t0 to tf with step size h
n = length(t);        % Number of time steps
x = zeros(1, n);      % Pre-allocate solution vector for x
x(1) = x0;            % Set the initial condition

% Loop through time steps and apply the RK4 method
for k = 1:(n-1)
    k1 = fun(t(k), x(k));                   % Compute the first slope estimate
    k2 = fun(t(k) + h/2, x(k) + h/2 * k1); % Compute the second slope estimate using k1
    k3 = fun(t(k) + h/2, x(k) + h/2 * k2); % Compute the third slope estimate using k2
    k4 = fun(t(k+1), x(k) + h * k3);        % Compute the fourth slope estimate using k3

    % Update the solution using the weighted average of all slope estimates
    x(k+1) = x(k) + (h/6) * (k1 + 2*k2 + 2*k3 + k4);
end
end
```

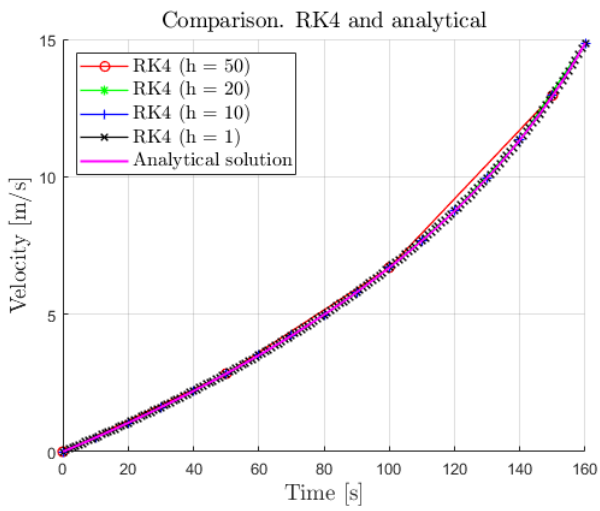


Figure 10: Comparison. RK4 vs analytical solution.

Summary of Observations

- **Improved Accuracy of RK4:** RK4 provides significantly better accuracy than RK2 (Heun method), especially for larger time steps. As the time step decreases, RK4 more closely approximates the analytical solution.

- **Comparison of Methods:**
  - For $h = 50$, RK4 shows moderate deviations but performs better than RK2.
  - For $h = 1$, RK4 achieves near-perfect agreement with the analytical solution, capturing finer details and handling stiffness effectively.
- **Key Insight:** RK4 is more robust and accurate across a range of time steps, making it ideal for problems requiring precision and reliability.

Table 3: Accuracy Comparison of RK4 for Different Time Steps h

Time Step h	Observations	Accuracy Level
50	Moderate deviations	Moderate
20	Minor discrepancies	High
10	Very minor discrepancies	Very High
1	Excellent agreement with analytical solution	Excellent

Conclusion: RK4 outperforms RK2 in accuracy and stability for solving differential equations. Even with larger time steps, RK4 produces reliable results. For $h = 1$, it closely matches the analytical solution, confirming its suitability for dynamically complex and stiff problems.

4. Trade off between CPU time & integration error.

   There are several options for calculating the integration error, such as Maximum error ($L\infty$), Mean Squared Error (MSE), Mean Absolute Error (MAE), Relative Root Mean Square Error (RRMSE), L2 Norm (Root of MSE).

   For the integration error, the MSE is used.

$$\text{MSE} = \frac{1}{N}\sum_{j=1}^{N}\left(y_{\text{numerical}}(t_j) - y_{\text{analytic}}(t_j)\right)^2 \tag{9}$$

   To calculate the CPU time we use the matlab function tic - toc. (Consider that CPU time varies as it depends on the characteristics of the computer, applications running in the background, etc.)

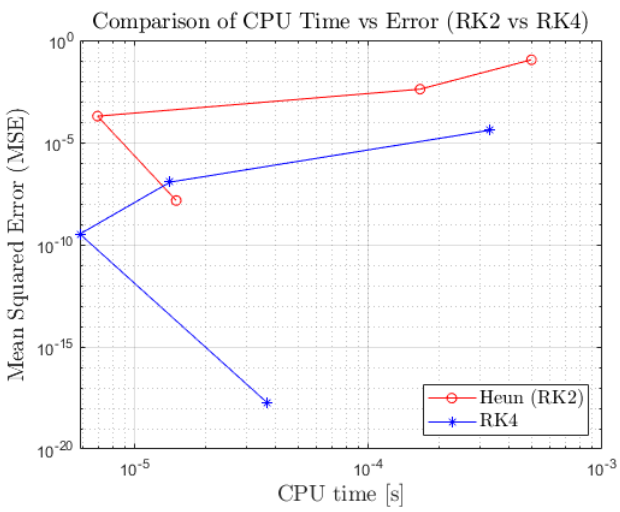


Figure 11: Comparison of CPU Time vs Error (RK2 vs RK4).

This plot highlights the trade-off between CPU time and mean squared error (MSE) for Heun's method (RK2) and RK4. Key observations include:

- As the step size h decreases, both methods reduce their integration error.
- RK4 consistently achieves a significantly lower error compared to RK2 for the same CPU time.

This demonstrates that RK4 is more computationally efficient in terms of accuracy per unit of CPU time, making it a better choice for applications requiring high precision.

Table 4: Integration error and CPU time for RK2 and RK4.

h	Integration error RK2	CPU time RK2 (s)	Integration error RK4	CPU time RK4 (s)
50	0.11122	0.0004977	3.98×10^{-5}	0.00033
20	0.0040438	0.000166	1.1462×10^{-7}	1.41×10^{-5}
10	0.00019383	6.9×10^{-6}	3.3092×10^{-10}	5.8×10^{-6}
1	1.4409×10^{-8}	1.5×10^{-5}	1.8951×10^{-18}	3.67×10^{-5}

Now, assuming that: the fluid density is $\rho = 900$ [kg/m^3], $C_d = 2.05$, and $A_m = 1$ [m^2], answer to the following tasks:

1. Solve Eq. (5) using RK2 in $t \in [0, 160]$s for $h_1 = 1$;

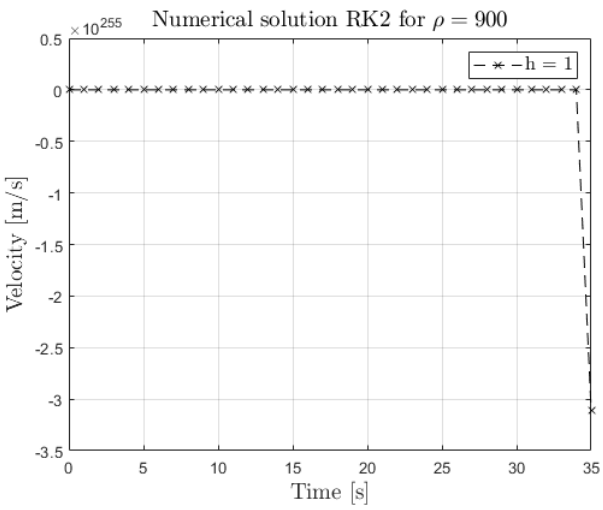


Figure 12: Numerical solution RK2 for $\rho = 900$ and h = 1.

Using Heun's method (RK2) with a step size $h = 1$ over the interval $t \in [0, 160]$ s, we observe that the numerical solution becomes unstable and diverges around $t = 35$ s, as indicated by the extremely large values of velocity ($v \sim 10^{255}$) (See figure 12).

This divergence likely occurs due to the rapid decrease in $m(t)$ over time as defined by $m(t) = m_0 - c_m \cdot t$. As t increases, $m(t)$ approaches zero, causing the differential equation to become stiff, meaning that small changes in v lead to disproportionately large changes in the solution.

Heun's method, a second-order Runge-Kutta method, lacks the numerical stability required to handle such stiff equations effectively, leading to an explosion in error that results in divergence.

2. From the results of the previous point, use a proper ode of Matlab to solve Eq. (5) ;

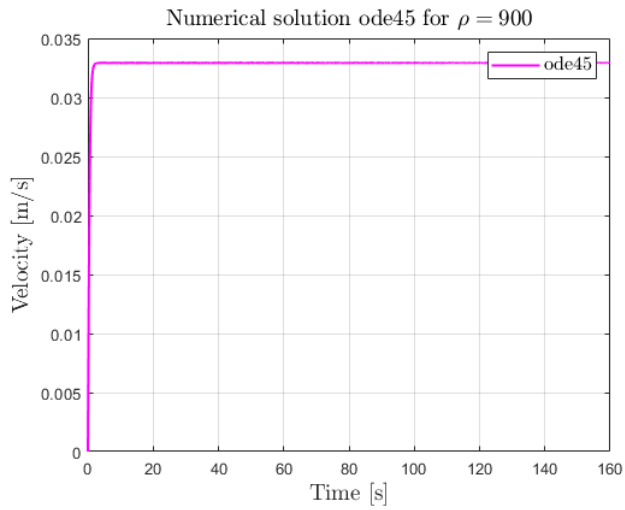
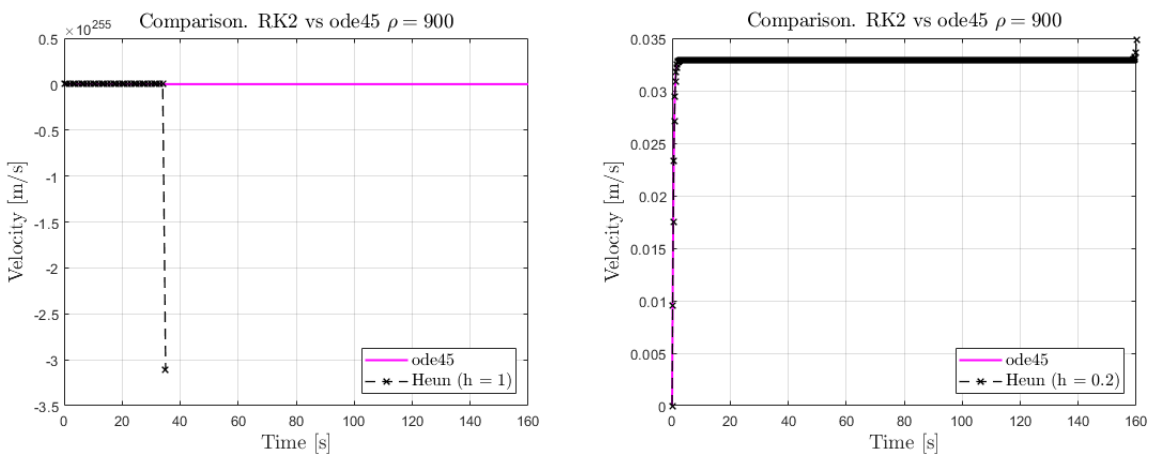


Figure 13: Numerical solution ode45 for $\rho = 900$.

Using MATLAB's `ode45`, the solution remains stable and accurate over $t \in [0, 160)$s (Figure 13). This adaptive solver dynamically adjusts step size using the Dormand-Prince method, effectively handling rapid changes or stiffness without sacrificing stability. While specialized solvers like `ode23s` or `ode15s` exist for stiff problems, `ode45` balances efficiency and accuracy for moderately stiff systems.

3. Discuss the results.

- RK2 struggles with stiffness in differential equations, especially when parameters like $m(t)$ vary significantly over time. Reducing h to 0.2 improves stability and allows RK2 to approximate the solution, but accuracy near the endpoint remains limited (see Figure 14), showing the need for adaptive solvers like `ode45`.
- `ode45` is stable across the entire interval and handles stiffness effectively, thanks to its ability to adapt the step size.
- Adaptive solvers like `ode45` are preferred for dynamically complex systems, ensuring accurate and reliable simulations without numerical instability.



(a) $h = 1$ **(b)** $h = 0.2$

Figure 14: Comparison of RK2 and ode45 for $\rho = 900$.

Exercise 3

Let $\dot{\mathbf{x}} = A(\alpha)\mathbf{x}$ be a two-dimensional system with $A(\alpha) = [0, 1; -1, 2\cos\alpha]$. Notice that $A(\alpha)$ has a pair of complex conjugate eigenvalues on the unit circle; α denotes the angle from the $\mathrm{Re}\{\lambda\}$-axis.

a) Runge-Kutta methods

1) Write the operator $F_{\mathrm{RK2}}(h, \alpha)$ that maps $\mathbf{x}_k$ into $\mathbf{x}_{k+1}$, namely $\mathbf{x}_{k+1} = F_{\mathrm{RK2}}(h, \alpha)\,\mathbf{x}_k$. 2) With $\alpha = \pi$, solve the problem "Find $h \geq 0$ s.t.$\max(|\mathrm{eig}(F(h, \alpha))|) = 1$". 3) Repeat point 2) for $\alpha \in [0, \pi]$ and draw the solutions in the $(h\lambda)$-plane. 4) Repeat points 1)–3) with RK4.

b) Backinterpolation methods

Consider the backinterpolation method $\mathrm{BI2}_{0.3}$. 1) Derive the expression of the linear operator $B_{\mathrm{BI2}_{0.3}}(h, \alpha)$ such that $\mathbf{x}_{k+1} = B_{\mathrm{BI2}_{0.3}}(h, \alpha)\mathbf{x}_k$. 2) Using the same approach of a), draw the stability domain of $\mathrm{BI2}_{0.3}$ in the $(h\lambda)$-plane. 3) Derive the domain of numerical stability of $\mathrm{BI2}_\theta$ for the values of $\theta = [0.2,\ 0.4,\ 0.6,\ 0.8]$.

(8 points)

---

a) Runge-Kutta methods

1. Write the operator $F_{\mathrm{RK2}}(h, \alpha)$ that maps $\mathbf{x}_k$ into $\mathbf{x}_{k+1}$, namely $\mathbf{x}_{k+1} = F_{\mathrm{RK2}}(h, \alpha)\,\mathbf{x}_k$. Repeat with RK4.

   - RK2

$$F_{\mathrm{RK2}}(h, \alpha) = I + hA(\alpha) + \frac{h^2}{2}A(\alpha)^2, \tag{10}$$

   - RK4

$$F_{\mathrm{RK4}}(h, \alpha) = I + hA(\alpha) + \frac{h^2}{2}A(\alpha)^2 + \frac{h^3}{6}A(\alpha)^3 + \frac{h^4}{24}A(\alpha)^4. \tag{11}$$

2. With $\alpha = \pi$, solve the problem "Find $h \geq 0$ s.t.$\max(|\mathrm{eig}(F(h, \alpha))|) = 1$".

   With $\alpha = \pi$, the matrix $A(\alpha)$ becomes:

$$A(\pi) = \begin{bmatrix} 0 & 1 \\ -1 & -2 \end{bmatrix}.$$

   We aim to find the largest value of $h \geq 0$ such that the eigenvalues of $F_{\mathrm{RKi}}(h, \alpha)$ satisfy:

$$\max(|\mathrm{eig}(F_{\mathrm{RKi}}(h, \alpha))|) = 1; i = 2, 4$$

   To achieve this, a bisection search is applied, adjusting h until the eigenvalue condition is met within the specified tolerance `tol_Ex3`.

3. Repeat point 2) for $\alpha \in [0, \pi]$ and draw the solutions in the $(h\lambda)$-plane.

   For this part, we repeat the procedure as in the last section over a range of α values from 0 to π. For each α, we calculate the maximum h that maintains stability, then plot the resulting values of $h\lambda$ in the complex plane.

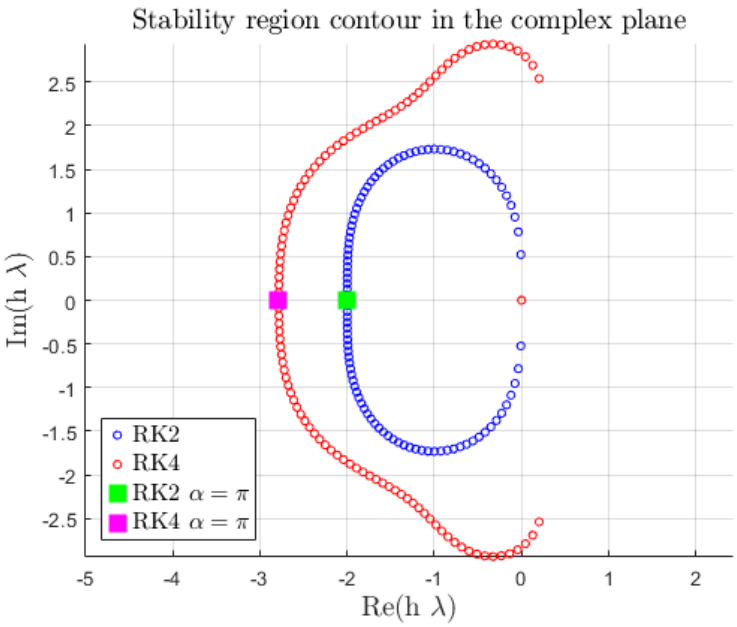


Figure 15: Stability region contour in the complex plane for RK2 and RK4 method.

b) Backinterpolation methods

Consider the backinterpolation method $\mathrm{BI2}_{0.3}$.

1. Derive the expression of the linear operator $B_{\mathrm{BI2}_{0.3}}(h,\alpha)$ such that $\mathbf{x}_{k+1} = B_{\mathrm{BI2}_{0.3}}(h,\alpha)\mathbf{x}_k$.

   The linear operator of the backinterpolation method $\mathrm{BI2}_\theta$ for a general parameter θ is defined as:

$$B_{\mathrm{BI2}_\theta}(h,\alpha) = \left(I - hA(\alpha)(1-\theta) + \frac{h^2}{2}A(\alpha)^2(1-\theta)^2\right)^{-1}\left(I + hA(\alpha)\theta + \frac{h^2}{2}A(\alpha)^2\theta^2\right).$$

   For $\theta = 0.3$, it becomes:

$$B_{\mathrm{BI2}_{0.3}}(h,\alpha) = \left(I - hA(\alpha)(0.7) + \frac{h^2}{2}A(\alpha)^2(0.7)^2\right)^{-1}\left(I + hA(\alpha)(0.3) + \frac{h^2}{2}A(\alpha)^2(0.3)^2\right).$$

2. Using the same approach as in part (a), draw the stability domain of $\mathrm{BI2}_{0.3}$ in the $(h\lambda)$-plane.

   Following the same bisection method used in part (a), we determine the maximum value of h for which the eigenvalues of $B_{\mathrm{BI2}_{0.3}}(h,\alpha)$ satisfy:

$$\max\left(|\mathrm{eig}(B_{\mathrm{BI2}_{0.3}}(h,\alpha))|\right) = 1.$$

   For each α value between 0 and π, we calculate h at the stability boundary and plot the values of $h\lambda$ in the complex plane.

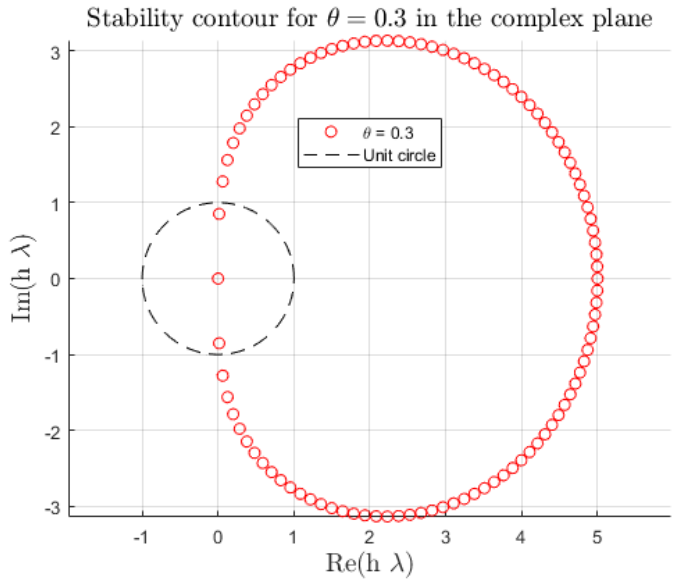


Figure 16: Stability region contour in the complex plane for the BI2 backinterpolation method with $\theta = 0.3$.

3. Derive the domain of numerical stability of BI2_θ for the values of $\theta = [0.2, 0.4, 0.6, 0.8]$.

   In this step, we repeat the stability analysis for the backinterpolation method BI2_θ for various θ values (0.2, 0.4, 0.6, and 0.8). The results are plotted in the complex plane in Figure 17. Each contour represents the stability boundary for a different θ value ($\theta = 0.3$ has also been included), highlighting the effect of varying θ on the stability region.

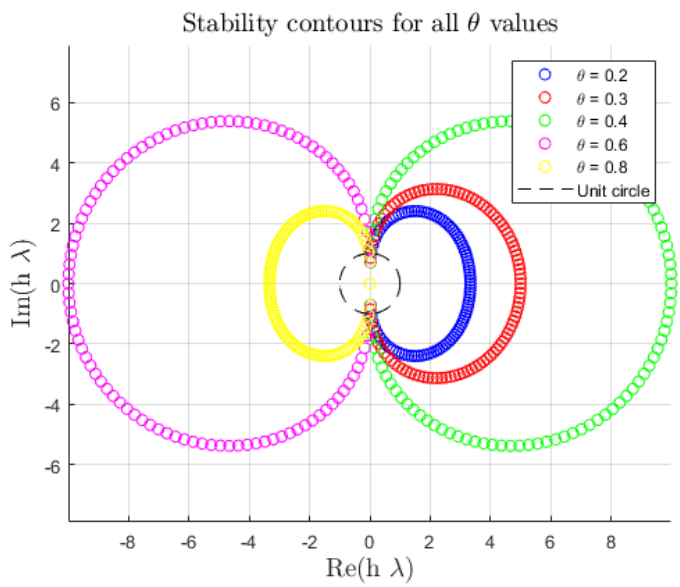


Figure 17: Stability contours for different θ values in the complex plane.

We observe that the stability region changes significantly with different values of θ. Smaller values of θ tend to result in larger stability regions, while higher values of θ reduce the region of stability in the complex plane. For values of θ less than 0.5 (right half-plane) the stable region is outside the circumference while for values of theta greater than 0.5 (left half-plane) the stable region is inside the circumference.

Exercise 4

The cooling problem of a high-temperature mass is shown in Fig. 18. Assuming that: K_c= 0.0042 [J/(s K)] and K_r = 6.15 x 10^{-11} [J/(s K^4)] are the convective and radiation heat loss coefficients respectively, $\widetilde{T}$ is the mass temperature in time, T_a = 277 [K] is the surrounding air temperature, C= 45 [J/K] is the mass thermal capacity, and $\widetilde{T}(0)$ = 555 [K] is the initial mass temperature.

1) Write the mathematical model of the system.
2) Solve the mathematical model using: RK2 with h= 720, and RK4 with h= 1440.
3) Compare the solution of point 2) with the "exact" one. Discuss the results. (Hint: use an ODE solver of Matlab).

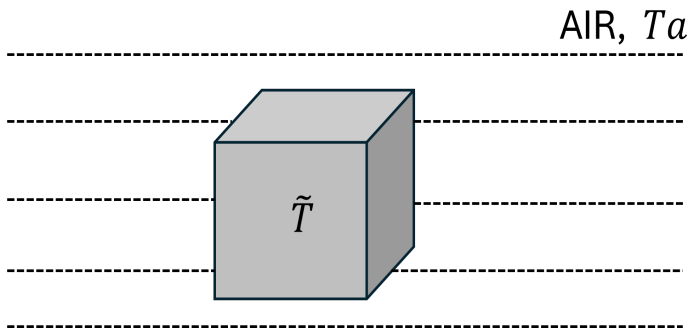


Figure 18: The cooling of high-temperature mass.

(4 points)

---

1. Write the mathematical model of the system.

   The temperature of the object $\tilde{T}(t)$ decreases over time due to heat loss through convection and radiation. The heat loss is governed by the following differential equation:

$$\frac{d\tilde{T}}{dt} = -\frac{1}{C}\left[K_c(\tilde{T} - T_a) + K_r(\tilde{T}^4 - T_a^4)\right] \tag{12}$$

   The initial condition is $\tilde{T}(0) = 555\,\mathrm{K}$, and we want to solve this equation over time until the temperature approaches ambient temperature T_a.

2. Solve the mathematical model using: RK2 with h= 720, and RK4 with h= 1440.

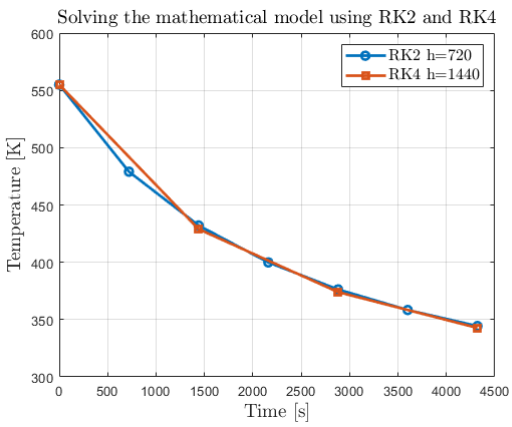
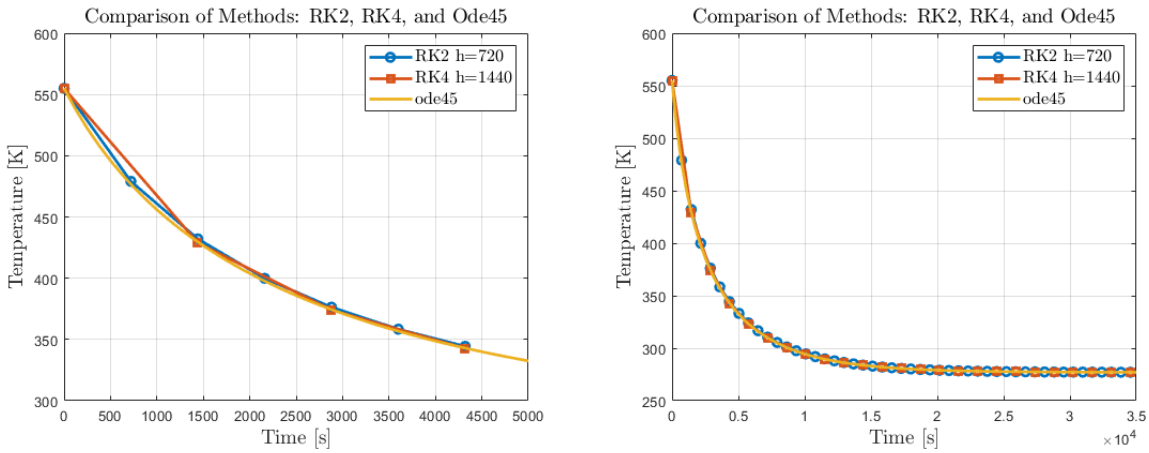


Figure 19: Solving the mathematical model using RK2 and RK4.

3. Compare the solution of point 2) with the "exact" one. Discuss the results. (Hint: use an ODE solver of Matlab).



(a) Initial cooling phase. **(b)** System is at ambient temperature.

Figure 20: Comparison of methods: RK2, RK4 and ode45.

In the figure 20, we compare the three methods used to solve the cooling model, RK2, RK4 and ode45 ("exact" solution).

The graph shows that:

- **RK2** and **RK4** follow the same general trend as `ode45`, which serves as the "exact" solution. However, **RK2** with a step size of $h = 720$ is less accurate compared to **RK4** with a step size of $h = 1440$ and `ode45`. This discrepancy is especially noticeable in the initial cooling phase, where the temperature drop is more rapid. Look at table 5
- **RK4** closely aligns with `ode45` throughout the simulation, even with a larger step size, demonstrating the higher accuracy of the fourth-order method.

Method	After 5000 seconds	After 35000 seconds
RK2	3.5233	0.4786
RK4	0.1894	0.0761

Table 5: MSE errors between the methods RK2 and RK4.

The values of 5000 seconds and 35000 seconds have been chosen. The value of 5000 was selected to observe that, initially, the difference is large because the temperature drops more quickly, and the value of 35000 was chosen because it is considered that the system is at ambient temperature; for this, it was verified that the temperature variation is less than 0.1 Kelvin degree.

Exercise 5

Consider the electrical circuit in Fig. 21. At time t>=0 the switch disconnects the battery and the capacitor discharges its stored energy to the circuit. Answer to the following tasks:

1) Find the state variable form of the circuit with x_1=q and x_2=$\frac{dq}{dt}$, where q is the charge on capacitor.

2) Show that the system is stiff when the circuit parameters are R=25 [Ω], L = 20 [mH], C = 200 [mF], and v_0 = 12 [V]. Represent the eigenvalues on the ($h\lambda$)-plane both for RK2 and IEX4 stability domain.

3) Use IEX4 to simulate the transient response of the system with the parameters of point 2), and determine the largest step size such that RK2 yields a stable and accurate solution.

4) Discuss the results.

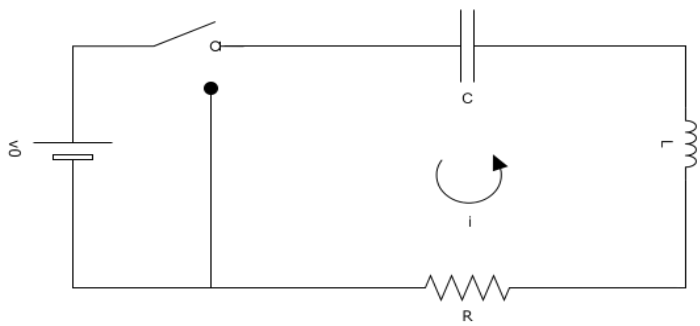


Figure 21: Electric circuit model.

(4 points)

---

1. Find the state variable form of the circuit with x_1=q and x_2=$\frac{dq}{dt}$, where q is the charge on capacitor.

   When the battery is disconnected at $t \geq 0$, the capacitor discharges, and the energy dissipates through R and L. Applying Kirchhoff's Voltage Law (KVL) to the loop:

$$v_L + v_R + v_C = 0$$

   where:

   - $v_L = L\frac{di}{dt}$ (voltage across the inductor),
   - $v_R = Ri$ (voltage across the resistor),
   - $v_C = \frac{q}{C}$ (voltage across the capacitor).

   The current, i, is related to the charge q on the capacitor as $i = \frac{dq}{dt}$. Define the state variables:

$$x_1 = q \quad \text{(charge on the capacitor)},$$
$$x_2 = \frac{dq}{dt} = i \quad \text{(current in the circuit)}.$$

From KVL, substituting these variables:

$$L\frac{di}{dt} + Ri + \frac{q}{C} = 0$$

In state-space form:

$$\frac{dx_1}{dt} = x_2,$$
$$\frac{dx_2}{dt} = -\frac{R}{L}x_2 - \frac{1}{LC}x_1.$$

Thus, the state-space representation is:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{1}{LC} & -\frac{R}{L} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

2. Show that the system is stiff when the circuit parameters are R=25 [Ω], L = 20 [mH], C = 200 [mF], and v_0 = 12 [V]. Represent the eigenvalues on the ($h\lambda$)-plane both for RK2 and IEX4 stability domain.

   To determine whether the system is stiff, we analyze its eigenvalues:

$$\lambda_{1,2} = -\frac{R}{2L} \pm \sqrt{\left(\frac{R}{2L}\right)^2 - \frac{1}{LC}}.$$

   For the given circuit parameters, the eigenvalues are:

$$\lambda_1 = -0.2000\, s^{-1}, \quad \lambda_2 = -1249.8000\, s^{-1}.$$

   The stiffness ratio is given by:

$$\text{Stiffness Ratio} = \frac{|\lambda_2|}{|\lambda_1|} \approx 6248.$$

   A system is considered stiff when it has eigenvalues with vastly different magnitudes, resulting in dynamics that evolve on very different timescales. In this case, the stiffness ratio indicates that the sistem has a slow component (λ_1) and a fast component (λ_2). This disparity in timescales confirms that the system is stiff.

   The figure below plots the eigenvalues in the ($h\lambda$)-plane for both the RK2 and IEX4 stability domains. I have selected a value of h = 0.01 so that the stability regions are clearly visible.

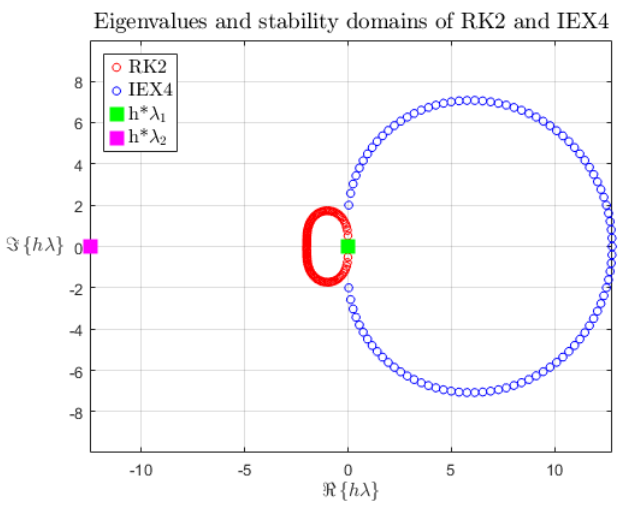


Figure 22: The eigenvalues in the ($h\lambda$)-plane for both the RK2 and IEX4 stability domains

3. Use IEX4 to simulate the transient response of the system with the parameters of point 2), and determine the largest step size such that RK2 yields a stable and accurate solution.

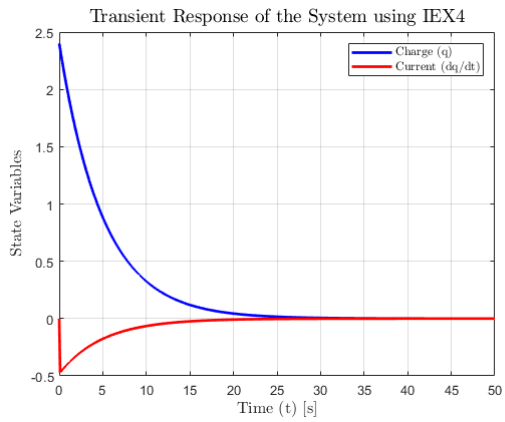


Figure 23: Simulation of the transient response of the system

The behavior shown in the graph is typical of a linear dynamic system that follows an exponential decay towards a steady-state.

- The **charge** (q) starts at a high value (~ 2.5) and decreases exponentially towards zero, indicating a gradual dissipation of energy in the system.
- The **current** ($\frac{dq}{dt}$) begins at a negative value (~ -0.5) and also converges smoothly to zero from below, reflecting the dissipation of the rate of change of charge.

The system is evidently stable under simulation using the IEX4 method, as no oscillations or divergences are observed in the solutions.

The largest step size such that RK2 yields a stable and accurate solution is $h_{max} = 0.0016$. This value is obtained so that $\lambda_2 h_{max} = 2$ (value that is in the contour of the RK2 method).

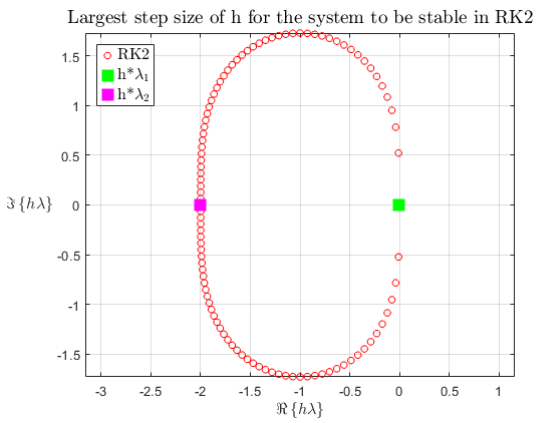


Figure 24: Largest step size of h for the system to be stable in RK2

4. Discuss the results.

   The IEX4 method has provided a stable and accurate solution, effectively capturing the system's transient response, even for relatively large step sizes (h). This behavior serves as a benchmark for assessing the performance of other numerical methods, such as RK2. For RK2, the maximum allowable step size must be determined to ensure both stability and accuracy. In our case, the maximum step size for stability is $h = 0.0016$.

Exercise 6

Consider the bouncing ball physical model in Fig. 25. The mathematical model of the system is represented by the following equations:

$$\frac{dv}{dt} = -g + \frac{F_D}{m_b} + \frac{F_A}{m_b} \tag{13}$$

$$\frac{dx}{dt} = v \tag{14}$$

$$v^+ = -k \cdot v^- \tag{15}$$

$$F_D = -\ 0.5 \cdot \rho \cdot C_d \cdot A_b \cdot v \cdot |v| \tag{16}$$

$$F_A = \rho \cdot V_b \cdot g \tag{17}$$

where the initial conditions are: $v(0) = 0$ [m/s], and $x(0) = 10$ [m].

Assuming that: the ball velocity before (v^-) and after (v^+) the ground impact is governed by the attenuation factor $k = 0.9$, the air density is $\rho = 1.225$ [kg/m^3], the ball mass is $m_b = 1$ [kg], the ball area is $A_b = 0.07$ [m^2], the ball volume is $V_b = 0.014$ [m^3], and the drag coefficient is C_d=1.17, answer to the following tasks:

1) Solve the mathematical model in the time interval $t \in [0, 10)$s using an ode integrator of Matlab.

2) Repeat point 1) for ρ=15 [kg/m^3]. Which is the difference with respect to point 1)?

3) Now solve the problem when the fluid density is ρ=60 [kg/m^3] in $t \in [0, 10)$s. Discuss the result.

4) Repeat point 3) using RK4 for $h = 3$. Discuss the result.

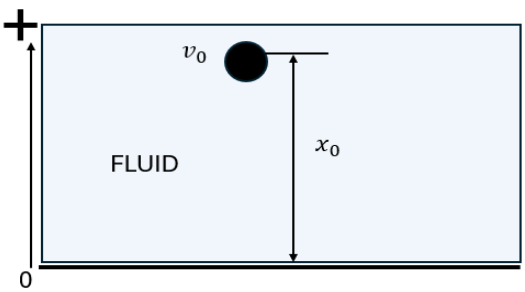


Figure 25: Bouncing ball system.

(5 points)

1. Solve the mathematical model in the time interval $t \in [0, 10)$s using an ode integrator of Matlab.

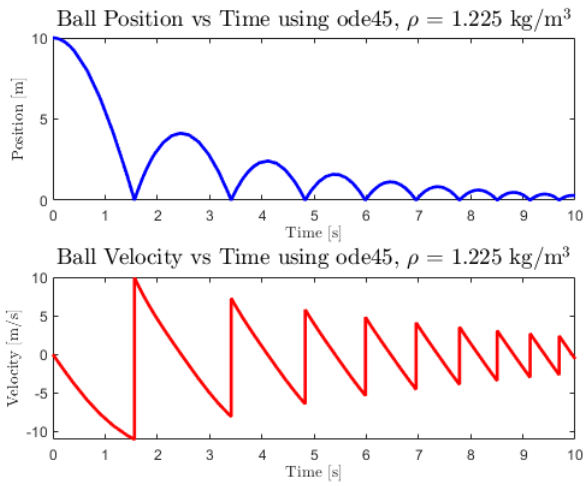


Figure 26: Ball position and velocity using ode45, $\rho = 1.225$ $[kg/m^3]$.

We solved the model over $t \in [0, 10)$ s using MATLAB's `ode45` solver. Its adaptive step-size adjusts to the solution's behavior, using larger steps during smooth phases like free fall and smaller steps to capture abrupt changes at impact points, balancing accuracy and efficiency.

The graph (Figure 26) shows:

- **Position vs Time**: The ball accelerates downward, bounces back with decreasing heights due to energy loss, modeled by a damping factor k.
- **Velocity vs Time**: The velocity alternates between negative (falling) and positive (rising), with reduced magnitudes after each bounce, reflecting the damping effect.

2. Repeat point 1) for ρ=15 [kg/m^3]. Which is the difference with respect to point 1)?

The increased fluid density ρ amplifies both the drag force ($F_D = -0.5 \cdot \rho \cdot C_d \cdot A_b \cdot v \cdot |v|$) and the buoyant force ($F_A = \rho \cdot V_b \cdot g$), causing the ball to lose speed more rapidly and "float" slightly more with each bounce. As a result, it loses energy faster, achieves lower rebound heights, and comes to rest sooner, as shown in Figure 27.

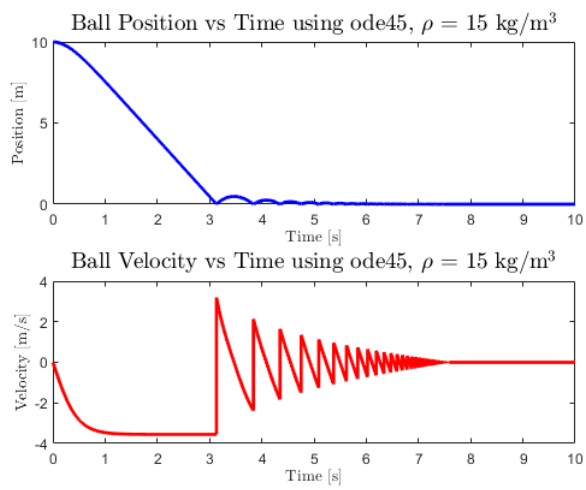


Figure 27: Ball position and velocity using ode45, $\rho = 15\,\mathrm{kg/m^3}$.

3. Now solve the problem when the fluid density is ρ=60 [kg/m^3] in $t \in [0, 10)$s. Discuss the result.

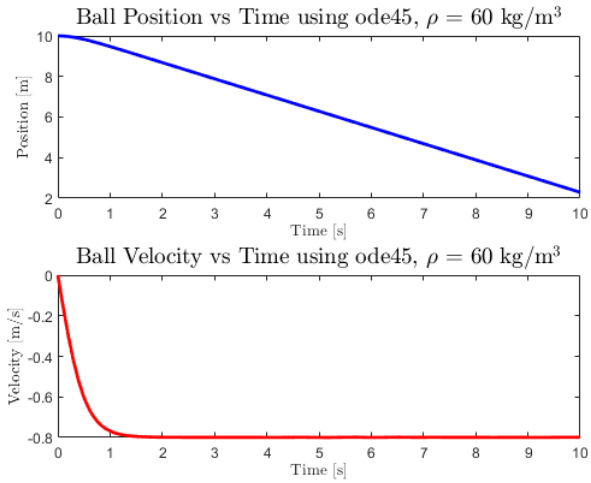


Figure 28: Ball position and velocity using ode45, $\rho = 60$ $[kg/m^3]$.

With $\rho = 60\,\mathrm{kg/m^3}$, the ball experiences strong drag and buoyant forces that nearly balance gravity, leading to a very slow descent without reaching the ground in 10 seconds. It quickly stabilizes at a near-zero terminal velocity, illustrating how high fluid density significantly reduces motion.

4. Repeat point 3) using RK4 for $h = 3$. Discuss the result.

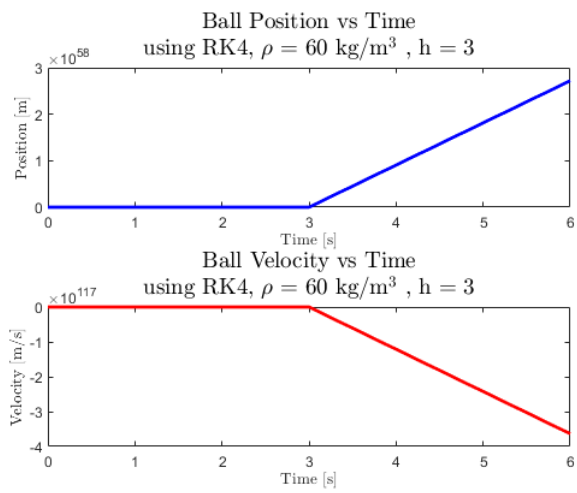
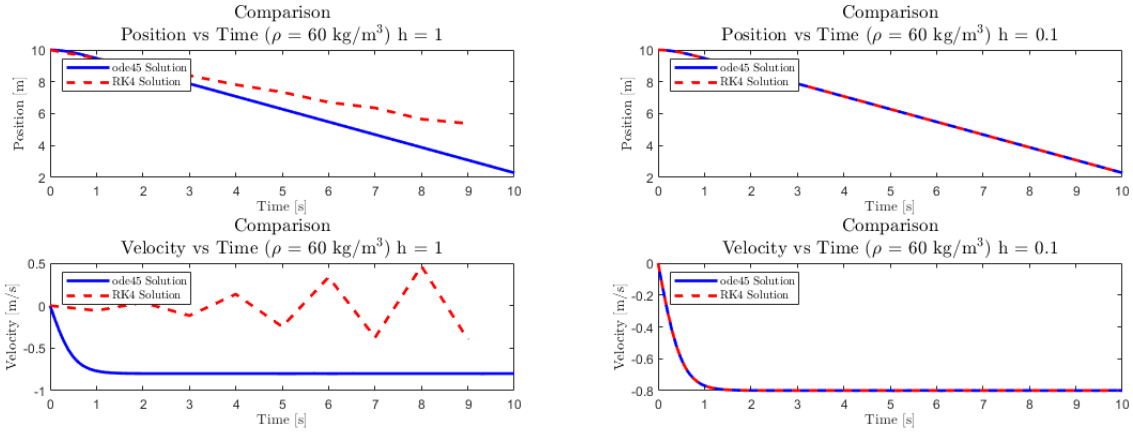


Figure 29: Ball position and velocity using RK4, $\rho = 60$ $[kg/m^3]$.

With $h = 3$, RK4 becomes unstable due to its inability to approximate the rapid nonlinear variations in λ, leading to nonsensical values like $h\lambda \approx -2.68 \times 10^{118}$. This instability occurs because the step size $h = 3$ is too large relative to the timescale on which the velocity and position of the ball change.



(a) h = 1. **(b)** h = 0.1.

Figure 30: Comparison ode45 vs RK4. Position and Velocity vs Time.

A smaller h resolves this:

- $h = 1$**:** The RK4 solution diverges slightly, with oscillations visible in velocity, indicating h is still too large.

- $h = 0.1$**:** RK4 closely matches `ode45`, with minimal errors demonstrating stable, accurate results.

- **Position MSE**: $8.84 \cdot 10^{-9}$
- **Velocity MSE**: $7.58 \cdot 10^{-8}$

This confirms that reducing h improves RK4's ability to capture rapid system changes, ensuring stability and reliability.